

# CS 6650 Assignment 4

Building Scalable Distributed Systems  
with ECS, Fargate & MapReduce

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Part II: Infrastructure Setup with ECS/Fargate</b> | <b>3</b>  |
| 1.1      | Overview                                              | 3         |
| 1.2      | Architecture Components                               | 3         |
| 1.2.1    | Amazon ECR (Elastic Container Registry)               | 3         |
| 1.2.2    | Amazon ECS (Elastic Container Service)                | 3         |
| 1.2.3    | AWS Fargate                                           | 3         |
| 1.3      | Deployment Steps                                      | 3         |
| 1.3.1    | Step 1: Build and Push Docker Image                   | 3         |
| 1.3.2    | Step 2: Create ECS Cluster                            | 3         |
| 1.3.3    | Step 3: Register Task Definition                      | 4         |
| 1.3.4    | Step 4: Run Task on Fargate                           | 4         |
| 1.3.5    | Step 5: Configure Security & Networking               | 4         |
| 1.4      | Screenshots                                           | 4         |
| 1.5      | Service Endpoints                                     | 6         |
| <b>2</b> | <b>Part III: MapReduce Implementation</b>             | <b>7</b>  |
| 2.1      | Architecture Design                                   | 7         |
| 2.2      | Implementation Details                                | 7         |
| 2.2.1    | Splitter Service (Go + Gin)                           | 7         |
| 2.2.2    | Mapper Services (3x Go + Gin)                         | 7         |
| 2.2.3    | Reducer Service (Go + Gin)                            | 8         |
| 2.3      | Execution Workflow                                    | 8         |
| 2.4      | Screenshots                                           | 9         |
| <b>3</b> | <b>Performance Analysis</b>                           | <b>11</b> |
| 3.1      | Experiment Design                                     | 11        |
| 3.2      | Results Summary                                       | 11        |
| 3.3      | Performance Graphs                                    | 11        |
| 3.4      | Distributed Processing Breakdown (160KB file)         | 12        |
| 3.5      | Key Findings                                          | 12        |
| 3.5.1    | Network Overhead Dominates                            | 12        |
| 3.5.2    | No Speedup for Small Files                            | 12        |
| 3.5.3    | Diminishing Overhead with Scale                       | 13        |
| 3.5.4    | When Would MapReduce Win?                             | 13        |
| 3.6      | Performance Insights for Production                   | 13        |
| <b>4</b> | <b>Key Learnings</b>                                  | <b>14</b> |
| 4.1      | Understanding Trade-offs                              | 14        |
| 4.2      | Real-World Applicability                              | 14        |
| 4.3      | Infrastructure as Code Benefits                       | 14        |
| 4.4      | Container Orchestration Value                         | 14        |
| <b>5</b> | <b>Exploration Questions</b>                          | <b>15</b> |
| 5.1      | EC2 vs ECS: What's the Difference?                    | 15        |
| 5.2      | What are VPC and Subnets?                             | 15        |
| 5.3      | TCP vs UDP: Protocol Comparison                       | 16        |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 5.4      | Controlling Task Resources . . . . .   | 16        |
| <b>6</b> | <b>Challenges Encountered</b>          | <b>18</b> |
| 6.1      | Service-Linked Role Issue . . . . .    | 18        |
| 6.2      | Docker Platform Architecture . . . . . | 18        |
| 6.3      | Go Module Dependencies . . . . .       | 18        |
| 6.4      | Network Latency . . . . .              | 18        |
| <b>7</b> | <b>Conclusion</b>                      | <b>19</b> |
| <b>A</b> | <b>Repository Structure</b>            | <b>20</b> |
| <b>B</b> | <b>Commands Reference</b>              | <b>20</b> |

# 1 Part II: Infrastructure Setup with ECS/Fargate

## 1.1 Overview

Deployed a Go-based REST API (albums service) to AWS using a modern container orchestration approach with ECR, ECS, and Fargate.

## 1.2 Architecture Components

### 1.2.1 Amazon ECR (Elastic Container Registry)

- **Purpose:** Store and version Docker images
- **Repository:** 498790386857.dkr.ecr.us-west-2.amazonaws.com/hello-service
- **Benefits:** Integrated with ECS, secure image storage, automatic scanning

### 1.2.2 Amazon ECS (Elastic Container Service)

- **Purpose:** Container orchestration and management
- **Cluster:** hello-cluster
- **Task Definition:** hello-task (0.25 vCPU, 0.5 GB memory)

### 1.2.3 AWS Fargate

- **Purpose:** Serverless compute for containers
- No EC2 instance management required
- Automatic scaling and resource allocation

## 1.3 Deployment Steps

### 1.3.1 Step 1: Build and Push Docker Image

```
1 # Build for AMD64 (Fargate requirement)
2 docker buildx build --platform linux/amd64 -t hello-service:latest .
3
4 # Tag for ECR
5 docker tag hello-service:latest $ECR_URL:latest
6
7 # Push to ECR
8 docker push $ECR_URL:latest
```

### 1.3.2 Step 2: Create ECS Cluster

```
1 aws ecs create-cluster --cluster-name hello-cluster --region us-west-2
```

### 1.3.3 Step 3: Register Task Definition

Created task definition with:

- Container image from ECR
- Port mapping: 8080
- Network mode: awsvpc
- Task and execution roles: LabRole

### 1.3.4 Step 4: Run Task on Fargate

```
1 aws ecs run-task \  
2 --cluster hello-cluster \  
3 --launch-type FARGATE \  
4 --task-definition hello-task:1 \  
5 --network-configuration "awsvpcConfiguration={...}"
```

### 1.3.5 Step 5: Configure Security & Networking

- Created security group allowing TCP port 8080
- Enabled auto-assign public IP
- Used default VPC and subnet

## 1.4 Screenshots

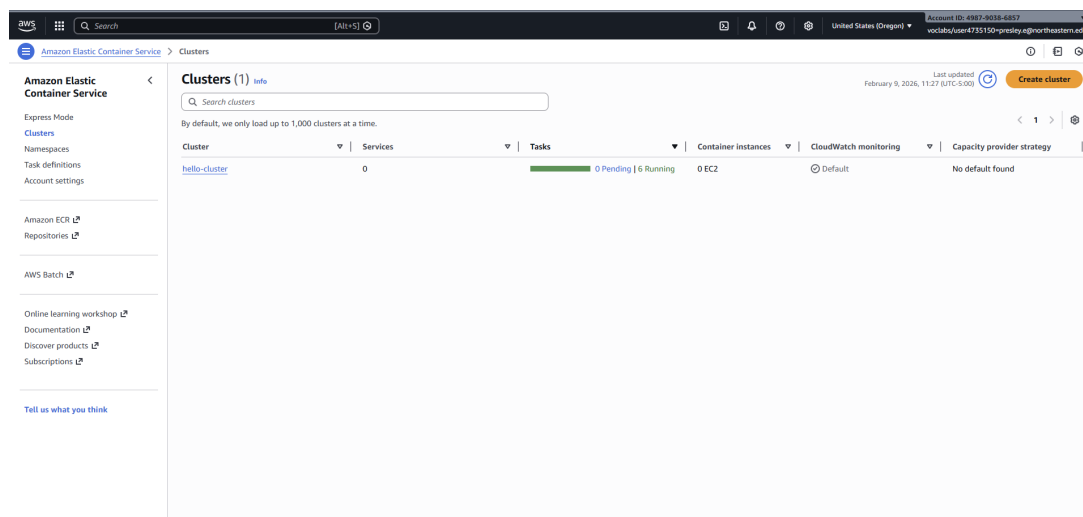


Figure 1: ECS cluster with running hello-service task

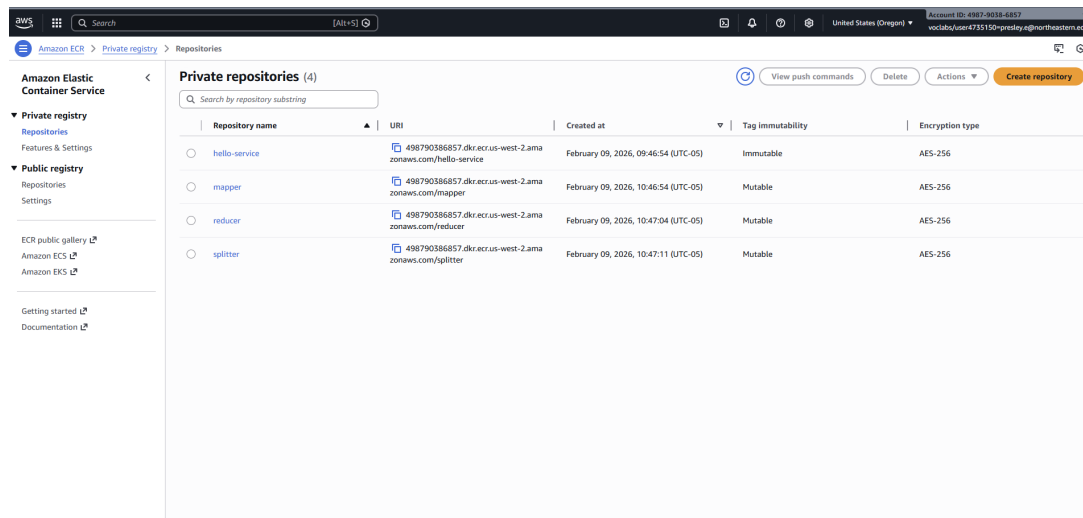


Figure 2: Docker image stored in ECR

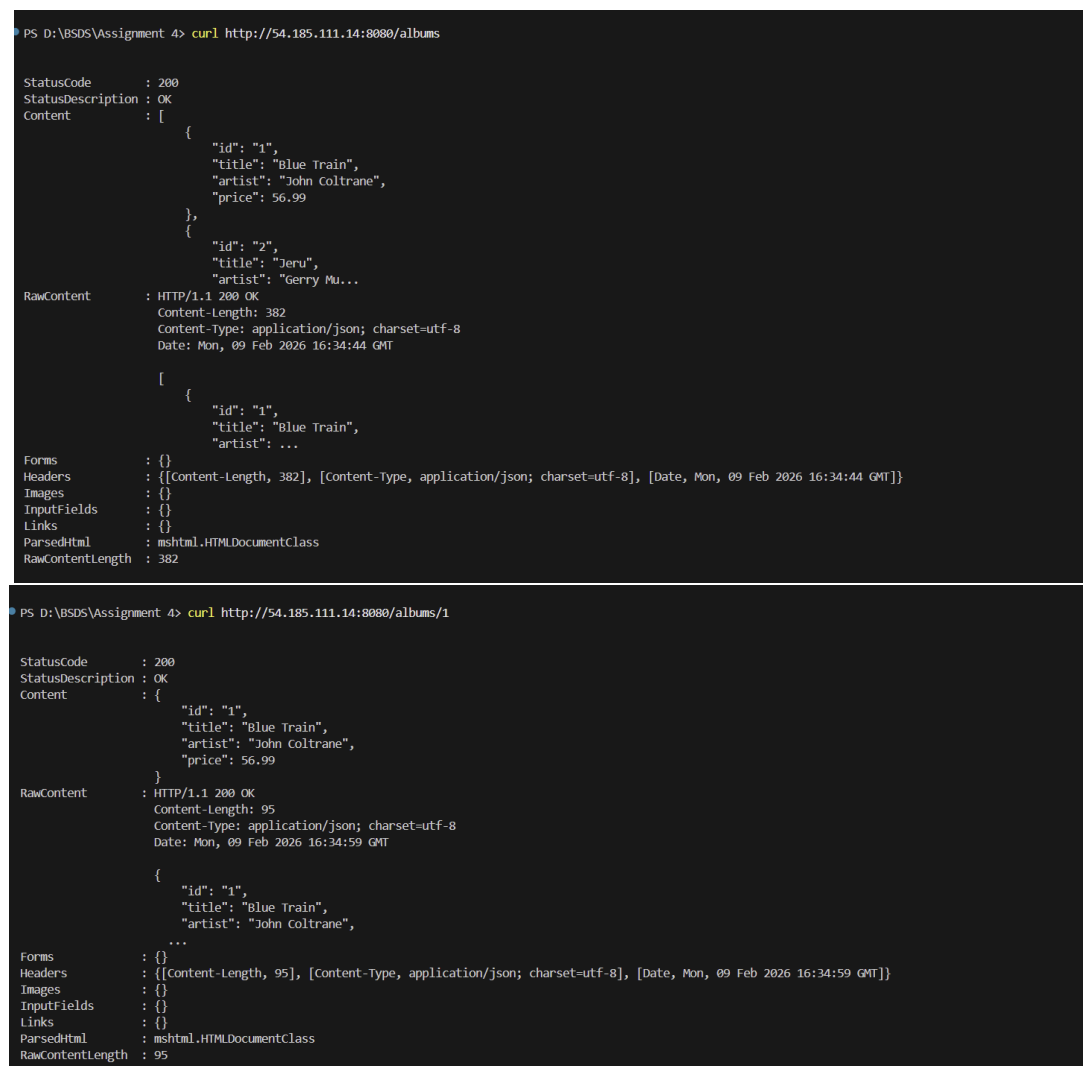


Figure 3: Successful curl tests showing album data

## 1.5 Service Endpoints

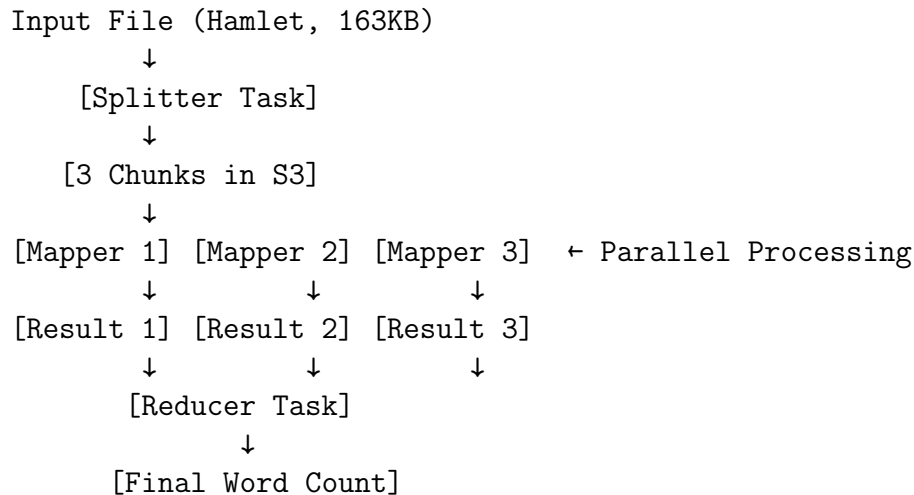
```
1 # Test get all albums
2 curl http://54.185.111.14:8080/albums
3
4 # Test get specific album
5 curl http://54.185.111.14:8080/albums/1
```

**Result:** Successfully deployed and accessed service running on AWS Fargate!

## 2 Part III: MapReduce Implementation

### 2.1 Architecture Design

Implemented a distributed word counting system using MapReduce pattern with 5 ECS tasks:



### 2.2 Implementation Details

#### 2.2.1 Splitter Service (Go + Gin)

**Purpose:** Split input file into equal chunks

**Key functionality:**

- Downloads file from S3
- Divides into N equal chunks
- Uploads chunks back to S3
- Returns S3 URLs of chunks

**Endpoint:** GET /split?bucket=X&key=Y

#### 2.2.2 Mapper Services (3x Go + Gin)

**Purpose:** Count words in assigned chunk

**Key functionality:**

- Downloads assigned chunk from S3
- Tokenizes and cleans words
- Counts word occurrences
- Uploads JSON result to S3
- Returns S3 URL of result

**Endpoint:** GET /map?bucket=X&key=Y&output=Z



### 2.2.3 Reducer Service (Go + Gin)

**Purpose:** Aggregate all mapper results

**Key functionality:**

- Downloads all mapper results from S3
- Merges word counts
- Produces final aggregated count
- Uploads final result to S3
- Returns total unique words

**Endpoint:** GET /reduce?bucket=X&mapper=Y1&mapper=Y2&mapper=Y3&output=Z

## 2.3 Execution Workflow

### Step 1: Split

```
1 curl "http://$SPLITTER_IP:8080/split?bucket=cs6650-mapreduce-66076&key=
  input.txt"
2 # Result: 3 chunks created in S3
```

### Step 2: Map (Parallel)

```
1 # Mapper 1 processes chunk-0
2 curl "http://$MAPPER1_IP:8080/map?bucket=...&key=chunk-0.txt&output=
  mapper-0-result.json"
3
4 # Mapper 2 processes chunk-1
5 curl "http://$MAPPER2_IP:8080/map?bucket=...&key=chunk-1.txt&output=
  mapper-1-result.json"
6
7 # Mapper 3 processes chunk-2
8 curl "http://$MAPPER3_IP:8080/map?bucket=...&key=chunk-2.txt&output=
  mapper-2-result.json"
```

### Step 3: Reduce

```
1 curl "http://$REDUCER_IP:8080/reduce?bucket=...&mapper=mapper-0-result.
  json&mapper=mapper-1-result.json&mapper=mapper-2-result.json&output=
  final-result.json"
2 # Result: 4,838 unique words
```

## 2.4 Screenshots

```

PS D:\BSDS\Assignment 4> curl "http://35.90.160.114:8080/split?bucket=cs6650-mapreduce-66076&key=input.txt"

StatusCode      : 200
StatusDescription : OK
Content         : [{"chunks":["s3://cs6650-mapreduce-66076/chunk-0.txt","s3://cs6650-mapreduce-66076/chunk-1.txt","s3://cs6650-mapreduce-66076/chunk-2.txt"]}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 138
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 09 Feb 2026 16:37:38 GMT

                  {"chunks":["s3://cs6650-mapreduce-66076/chunk-0.txt","s3://cs6650-mapreduce-...
Forms           : {}
Headers        : [{"Content-Length", 138}, [{"Content-Type", "application/json; charset=utf-8"}, [{"Date", "Mon, 09 Feb 2026 16:37:38 GMT"}]]
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshtml.HTMLDocumentClass
RawContentLength : 138

PS D:\BSDS\Assignment 4> curl "http://54.184.175.227:8080/map?bucket=cs6650-mapreduce-66076&key=chunk-0.txt&output=mapper-0-result.json"

StatusCode      : 200
StatusDescription : OK
Content         : {"result":"s3://cs6650-mapreduce-66076/mapper-0-result.json"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 61
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 09 Feb 2026 16:38:04 GMT

                  {"result":"s3://cs6650-mapreduce-66076/mapper-0-result.json"}
Forms           : {}
Headers        : [{"Content-Length", 61}, [{"Content-Type", "application/json; charset=utf-8"}, [{"Date", "Mon, 09 Feb 2026 16:38:04 GMT"}]]
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshtml.HTMLDocumentClass
RawContentLength : 61

```

Figure 4: MapReduce Step 1: Splitter execution showing 3 chunks created

```

PS D:\BSDS\Assignment 4> curl "http://16.144.225.40:8080/map?bucket=cs6650-mapreduce-66076&key=chunk-1.txt&output=mapper-1-result.json"

StatusCode      : 200
StatusDescription : OK
Content         : {"result":"s3://cs6650-mapreduce-66076/mapper-1-result.json"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 61
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 09 Feb 2026 16:38:11 GMT

                  {"result":"s3://cs6650-mapreduce-66076/mapper-1-result.json"}
Forms           : {}
Headers        : [{"Content-Length", 61}, [{"Content-Type", "application/json; charset=utf-8"}, [{"Date", "Mon, 09 Feb 2026 16:38:11 GMT"}]]
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshtml.HTMLDocumentClass
RawContentLength : 61

PS D:\BSDS\Assignment 4> curl "http://54.212.142.164:8080/map?bucket=cs6650-mapreduce-66076&key=chunk-2.txt&output=mapper-2-result.json"

StatusCode      : 200
StatusDescription : OK
Content         : {"result":"s3://cs6650-mapreduce-66076/mapper-2-result.json"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 61
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 09 Feb 2026 16:38:17 GMT

                  {"result":"s3://cs6650-mapreduce-66076/mapper-2-result.json"}
Forms           : {}
Headers        : [{"Content-Length", 61}, [{"Content-Type", "application/json; charset=utf-8"}, [{"Date", "Mon, 09 Feb 2026 16:38:17 GMT"}]]
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshtml.HTMLDocumentClass
RawContentLength : 61

```

Figure 5: MapReduce Step 2: Three mappers processing chunks in parallel

```

PS D:\8SDS\Assignment 4> curl "http://54.202.52.92:8080/reduce?bucket=cs6650-mapreduce-66076&mapper=mapper-0-result.json&mapper=mapper-1-result.json&mapper=map
per-2-result.json&output=final-result.json"

StatusCode      : 200
StatusDescription : OK
Content          : {"result":"s3://cs6650-mapreduce-66076/final-result.json","total_words":4838}
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 77
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 09 Feb 2026 16:38:23 GMT

{"result":"s3://cs6650-mapreduce-66076/final-result.json","total_words":4838}...

Forms           : {}
Headers          : [[Content-Length, 77], [Content-Type, application/json; charset=utf-8], [Date, Mon, 09 Feb 2026 16:38:23 GMT]]
Images           : {}
InputFields      : {}
Links            : {}
Parsedhtml       : mshtml.HTMLDocumentClass
RawContentLength : 77

```

Figure 6: MapReduce Step 3: Reducer aggregating results (4,838 unique words)

| Name                       | Type | Last modified                          | Size     | Storage class |
|----------------------------|------|----------------------------------------|----------|---------------|
| chunk-1.txt                | txt  | February 9, 2026, 11:37:39 (UTC-05:00) | 53.0 KB  | Standard      |
| chunk-2.txt                | txt  | February 9, 2026, 11:37:39 (UTC-05:00) | 53.0 KB  | Standard      |
| final-10x.json             | json | February 9, 2026, 11:00:15 (UTC-05:00) | 59.7 KB  | Standard      |
| final-50x.json             | json | February 9, 2026, 11:00:18 (UTC-05:00) | 61.3 KB  | Standard      |
| final-result-timed.json    | json | February 9, 2026, 10:58:11 (UTC-05:00) | 55.0 KB  | Standard      |
| final-result.json          | json | February 9, 2026, 11:38:24 (UTC-05:00) | 55.0 KB  | Standard      |
| input-10x.txt              | txt  | February 9, 2026, 10:59:47 (UTC-05:00) | 1.6 MB   | Standard      |
| input-50x.txt              | txt  | February 9, 2026, 10:59:52 (UTC-05:00) | 8.0 MB   | Standard      |
| input.txt                  | txt  | February 9, 2026, 10:30:26 (UTC-05:00) | 159.1 KB | Standard      |
| mapper-0-10x.json          | json | February 9, 2026, 11:00:14 (UTC-05:00) | 55.7 KB  | Standard      |
| mapper-0-50x.json          | json | February 9, 2026, 11:00:17 (UTC-05:00) | 59.9 KB  | Standard      |
| mapper-0-result-timed.json | json | February 9, 2026, 10:58:10 (UTC-05:00) | 24.4 KB  | Standard      |
| mapper-0-result.json       | json | February 9, 2026, 11:38:05 (UTC-05:00) | 24.4 KB  | Standard      |
| mapper-1-10x.json          | json | February 9, 2026, 11:00:15 (UTC-05:00) | 55.8 KB  | Standard      |
| mapper-1-50x.json          | json | February 9, 2026, 11:00:17 (UTC-05:00) | 59.9 KB  | Standard      |
| mapper-1-result-timed.json | json | February 9, 2026, 10:58:11 (UTC-05:00) | 26.3 KB  | Standard      |
| mapper-1-result.json       | json | February 9, 2026, 11:38:12 (UTC-05:00) | 26.3 KB  | Standard      |
| mapper-2-10x.json          | json | February 9, 2026, 11:00:15 (UTC-05:00) | 55.7 KB  | Standard      |
| mapper-2-50x.json          | json | February 9, 2026, 11:00:18 (UTC-05:00) | 59.9 KB  | Standard      |
| mapper-2-result-timed.json | json | February 9, 2026, 10:58:11 (UTC-05:00) | 25.1 KB  | Standard      |
| mapper-2-result.json       | json | February 9, 2026, 11:38:18 (UTC-05:00) | 25.1 KB  | Standard      |

Figure 7: Input files, chunks, mapper results, and final output in S3

| Task                     | Last status | Desired status | Task definition | Health status | Created at     | Started by | Started at     | Group        |
|--------------------------|-------------|----------------|-----------------|---------------|----------------|------------|----------------|--------------|
| 0b0b7052c1ec4b389992...  | Running     | Running        | reducer-task:1  | Unknown       | 52 minutes ago | -          | 52 minutes ago | family:redu  |
| 80x42d9b2a3b4016b80cf... | Running     | Running        | mapper-task:1   | Unknown       | 52 minutes ago | -          | 52 minutes ago | family:map   |
| ab1cd7856d01426d90a2...  | Running     | Running        | splitter-task:1 | Unknown       | 53 minutes ago | -          | 52 minutes ago | family:split |
| b08a8fc779eb490387a32... | Running     | Running        | hello-task:1    | Unknown       | 2 hours ago    | -          | 2 hours ago    | family:hello |
| efc0b43561a64e7391c1d... | Running     | Running        | mapper-task:1   | Unknown       | 52 minutes ago | -          | 52 minutes ago | family:map   |
| f493ffae177477488956b... | Running     | Running        | mapper-task:1   | Unknown       | 52 minutes ago | -          | 52 minutes ago | family:map   |

Figure 8: 6 tasks running: 1 hello-service + 5 MapReduce tasks

### 3 Performance Analysis

#### 3.1 Experiment Design

Tested three file sizes with both sequential and distributed processing:

- **Small:** Hamlet 1x (160KB)
- **Medium:** Hamlet 10x (1.6MB)
- **Large:** Hamlet 50x (8MB)

#### 3.2 Results Summary

| File Size | Sequential | Distributed | Overhead | Overhead % |
|-----------|------------|-------------|----------|------------|
| 160KB     | 0.017s     | 1.528s      | 1.511s   | 98.9%      |
| 1.6MB     | 0.076s     | 2.089s      | 2.013s   | 96.4%      |
| 8MB       | 0.402s     | 2.673s      | 2.271s   | 85.0%      |

Table 1: Performance comparison: Sequential vs Distributed processing

#### 3.3 Performance Graphs

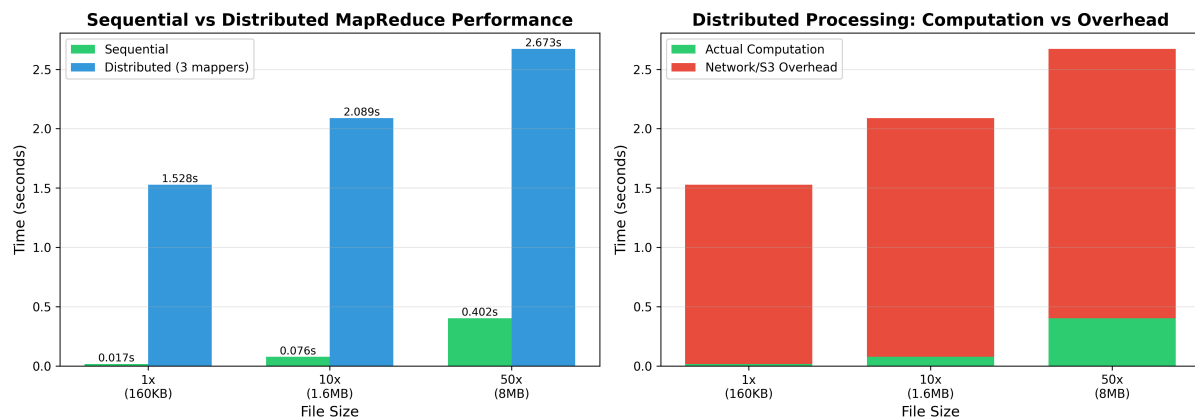


Figure 9: Sequential vs Distributed processing times across different file sizes

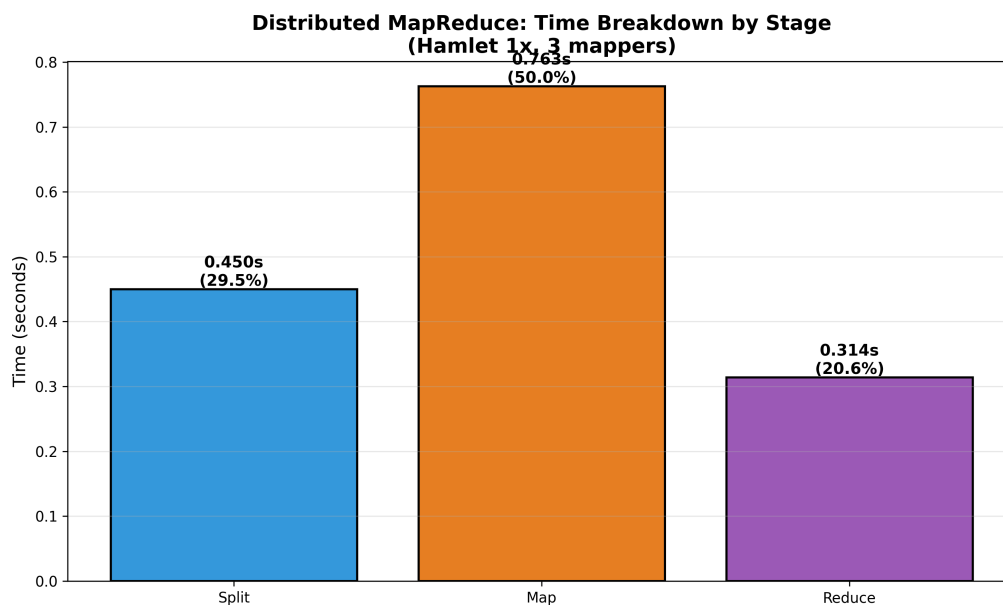


Figure 10: Time breakdown by MapReduce stage (Split, Map, Reduce)

### 3.4 Distributed Processing Breakdown (160KB file)

- **Split Time:** 0.450s (29.4%)
- **Map Time:** 0.763s (49.9%)
- **Reduce Time:** 0.314s (20.5%)
- **Total Time:** 1.528s

### 3.5 Key Findings

#### 3.5.1 Network Overhead Dominates

For small to medium files, the overhead of network communication and S3 operations far exceeds the actual computation time. The  $\sim 1.5$  second baseline overhead includes:

- HTTP requests to each service
- S3 download/upload operations (6 uploads, 7 downloads)
- Container startup and initialization
- JSON serialization/deserialization

#### 3.5.2 No Speedup for Small Files

**Speedup Ratio:** 0.01x (90x slower!)

For the 160KB Hamlet file:

- Sequential: 0.017s
- Distributed: 1.528s

The distributed version is **90 times slower** because the overhead (1.511s) dwarfs the actual computation (0.017s).

### 3.5.3 Diminishing Overhead with Scale

As file size increases, overhead becomes a smaller percentage:

- 160KB: 98.9% overhead
- 1.6MB: 96.4% overhead
- 8MB: 85.0% overhead

### 3.5.4 When Would MapReduce Win?

Based on our measurements, distributed MapReduce would be beneficial when:

- **File size > 100MB+:** Overhead becomes negligible compared to computation
- **Complex computations:** More than simple word counting (e.g., NLP, machine learning)
- **Many mappers:** Scaling to 10+ mappers for truly large datasets
- **Real-world scenarios:** Processing terabytes of log files, analyzing large datasets

## 3.6 Performance Insights for Production

**Strengths of this approach:**

Scalability: Can add more mappers for larger files

Fault tolerance: Individual tasks can be restarted

No infrastructure management: Fargate handles compute

Parallelization: Multiple mappers work simultaneously

**Limitations observed:**

- × High baseline overhead (~1.5s)
- × S3 latency for small operations
- × HTTP request overhead per service call
- × Not cost-effective for small files

**Optimizations to consider:**

- Batch multiple operations to reduce round trips
- Use AWS Lambda for smaller tasks (faster cold start)
- Implement caching for repeated operations
- Use larger chunk sizes to reduce coordination overhead
- Consider streaming data instead of S3 for intermediate results

## 4 Key Learnings

### 4.1 Understanding Trade-offs

This assignment demonstrated that **distributed**  $\neq$  **faster**. The trade-off is:

- **Distributed systems:** Higher overhead, better scalability for large workloads
- **Sequential processing:** Lower overhead, limited by single machine capacity

### 4.2 Real-World Applicability

My experience at Capgemini with Apache Beam now makes more sense - we processed **terabytes** of data where the overhead was negligible compared to computation. For smaller datasets, we often used simpler approaches.

### 4.3 Infrastructure as Code Benefits

Using AWS CLI and task definitions provided:

- Reproducibility: Can recreate entire setup with commands
- Version control: Infrastructure changes can be tracked
- Automation potential: Could automate scaling and deployment

### 4.4 Container Orchestration Value

ECS + Fargate eliminated:

- Server management overhead
- Capacity planning complexity
- Manual scaling operations

## 5 Exploration Questions

### 5.1 EC2 vs ECS: What's the Difference?

#### EC2 (Elastic Compute Cloud):

- Virtual machines in the cloud
- **You manage:** OS, runtime, application, scaling
- **Use when:** Need full control, specific OS requirements, non-containerized apps

#### ECS (Elastic Container Service):

- Container orchestration service
- **You manage:** Container definitions, task configurations
- **AWS manages:** Scheduling, placement, health monitoring
- **Use when:** Running containerized applications, need auto-scaling, want less infrastructure management

**In this assignment:** We used ECS with Fargate (serverless), so we only defined what to run (containers), not where to run it (servers).

### 5.2 What are VPC and Subnets?

#### VPC (Virtual Private Cloud):

- Isolated virtual network in AWS
- Like having your own private data center in the cloud
- Controls IP addressing, routing, security

#### Default VPC:

- AWS provides one automatically in each region
- Pre-configured with internet gateway, subnets, route tables
- We used: `vpc-0d3cc528441a3a27c`

#### Subnet:

- Subdivision of VPC's IP address range
- Can be public (internet-accessible) or private (internal only)
- Spans a single Availability Zone
- We used: `subnet-065c644092af8455f` (public subnet in us-west-2c)

#### How we got access:

1. AWS Academy Learner Lab provides pre-configured default VPC
2. Our tasks were launched in a public subnet
3. Enabled `assignPublicIp=ENABLED` to get internet-routable IP addresses
4. Security group allowed inbound traffic on port 8080



## 5.3 TCP vs UDP: Protocol Comparison

### TCP (Transmission Control Protocol):

- **Connection-based:** Establishes connection before sending data (3-way handshake)
- **Reliable:** Guarantees data delivery and order
- **Error checking:** Detects and retransmits lost packets
- **Flow control:** Adjusts transmission rate based on network conditions
- **Use cases:** HTTP, HTTPS, SSH, email
- **Our use:** HTTP REST APIs run on TCP

### UDP (User Datagram Protocol):

- **Connectionless:** No handshake, just sends packets
- **Unreliable:** No delivery guarantee, packets may be lost or arrive out of order
- **Faster:** Lower overhead, no acknowledgment required
- **Use cases:** Video streaming, online gaming, DNS queries
- **Trade-off:** Speed over reliability

**Why we used TCP:** REST APIs require reliable delivery - we can't afford to lose HTTP requests or responses.

## 5.4 Controlling Task Resources

In ECS Task Definition, we specified:

```
1 {  
2   "cpu": "256",           // 0.25 vCPU  
3   "memory": "512",       // 0.5 GB RAM  
4   ...  
5 }
```

### CPU Units:

- 1024 units = 1 vCPU
- We used 256 units = 0.25 vCPU per task
- Total for 6 tasks: 1.5 vCPUs

### Memory:

- Specified in MB
- We used 512 MB per task
- Total for 6 tasks: 3 GB

**Why these values:**

- Word counting is lightweight (not CPU/memory intensive)
- Minimizes costs during testing
- Can scale up if processing larger files or complex operations

**Fargate Resource Options:**

Limited to specific CPU/memory combinations:

- 0.25 vCPU: 0.5GB, 1GB, 2GB
- 0.5 vCPU: 1GB to 4GB
- 1 vCPU: 2GB to 8GB
- 2 vCPU: 4GB to 16GB
- 4 vCPU: 8GB to 30GB

## 6 Challenges Encountered

### 6.1 Service-Linked Role Issue

**Problem:** ECS cluster creation failed with "Unable to assume service linked role"

**Solution:** Created service-linked role using AWS CLI:

```
1 aws iam create-service-linked-role --aws-service-name ecs.amazonaws.com
```

### 6.2 Docker Platform Architecture

**Problem:** Built images on Windows couldn't run on Fargate (AMD64 required)

**Solution:** Used `docker buildx` with `--platform linux/amd64` flag

### 6.3 Go Module Dependencies

**Problem:** Missing `go.sum` entries caused Docker build failures

**Solution:** Ran `go mod tidy` locally before Docker build to generate complete dependency list

### 6.4 Network Latency

**Problem:** High overhead in distributed processing

**Learning:** Confirmed that network/S3 operations dominate for small workloads

## 7 Conclusion

This assignment provided hands-on experience with:

1. **Modern container orchestration** using ECS and Fargate
2. **Distributed computing patterns** through MapReduce implementation
3. **Performance trade-offs** between sequential and distributed processing
4. **Cloud infrastructure** management using AWS services

**Key Takeaway:** Distributed systems are powerful tools, but they come with overhead. Understanding when to use them (large-scale data, complex computations) versus when simpler approaches suffice (small datasets, simple operations) is crucial for building efficient systems.

The hands-on implementation reinforced concepts from the course readings and connected directly to my professional experience with distributed data pipelines at Capgemini.

## A Repository Structure

```
Assignment 4/  
  splitter/  
    main.go  
    go.mod  
    go.sum  
    Dockerfile  
  mapper/  
    main.go  
    go.mod  
    go.sum  
    Dockerfile  
  reducer/  
    main.go  
    go.mod  
    go.sum  
    Dockerfile  
  task definitions (JSON files)  
  performance test scripts (Python)  
  graphs (PNG files)  
  this report
```

## B Commands Reference

### AWS Configuration:

```
1 aws configure  
2 aws configure set aws_session_token <TOKEN>
```

### ECR Operations:

```
1 aws ecr create-repository --repository-name <name>  
2 docker push <ECR_URL>:latest
```

### ECS Operations:

```
1 aws ecs create-cluster --cluster-name <name>  
2 aws ecs register-task-definition --cli-input-json file://task.json  
3 aws ecs run-task --cluster <cluster> --task-definition <task>
```

### S3 Operations:

```
1 aws s3 mb s3://<bucket-name>  
2 aws s3 cp <file> s3://<bucket>/<key>  
3 aws s3 ls s3://<bucket>/
```